

# جزوه امتحانی – داده‌های حجیم (Big Data Analytics)

استاد: دکتر فرساد زمانی بروجنی | امتحان: جزوه‌باز – ۹۰ دقیقه | منبع: لکچرهای ترم + نمونه سوالات + MMDS

**روش پاسخ:** تعریف یک خطی ← فرمول داخل باکس ← مثال عددی کوتاه. اول **فهرست صفحات** و **چیت‌شیت** را ببین. اگر سوال شبیه نمونه استاد بود → بخش 10.

## فهرست صفحات (شماره واقعی همین PDF) [INDEX]

در امتحان برو به **شماره صفحه** ستون آخر. این شماره‌ها با پاصفحه PDF یکی هستند.

| کلمه کلیدی سوال                         | بخش     | چه بنویسی            | صفحه |
|-----------------------------------------|---------|----------------------|------|
| چیت‌شیت و فرمول‌های حیاتی               | چیت‌شیت | تعریف + فرمول        | 1    |
| TF-IDF / Collaborative Filtering        | بخش 1   | فرمول + مثال عددی    | 2    |
| Cluster ↔ RDBMS / Hadoop                | بخش 2   | جدول تفاوت‌ها        | 3    |
| MapReduce / Combiner / Pregel           | بخش 3   | ۳ مرحله + خرابی      | 6    |
| Join / جبر رابطه‌ای                     | بخش 4   | مثال Join            | 7    |
| Matrix × Vector                         | بخش 5   | Map/Reduce + عدد     | 7    |
| Shingle / Jaccard / MinHash / LSH / PCY | بخش 6   | تعریف + فرمول + مثال | 10   |
| فاصله‌ها / اسمی / دودویی                | بخش 7   | فرمول + مثال         | 12   |
| Apriori / Closed / Maximal              | بخش 8   | M ≤ C ≤ F + ها Pass  | 13   |
| Clustering / K-Means / DBSCAN / BFR     | بخش 9   | مثال یک تکرار        | 15   |
| حفظیات کامل KDD / مخازن / Hadoop        | تکمله A | تعریف‌های جزئی       | 4    |
| اجرای داخلی / هزینه / Multi-way Join    | تکمله B | جزئیات MapReduce     | 8    |
| MinHash / LSH / CHARM با جزئیات         | تکمله C | اثبات و دام‌ها       | 13   |
| الگوریتم‌های خوشه‌بندی کامل             | تکمله D | فرض/مراحل/مقایسه     | 16   |
| حل نمونه سوالات استاد                   | بخش 10  | پاسخ آماده           | 19   |

کل صفحات PDF: 20 | چیت‌شیت → ص 1 | بخش 1 → ص 2 · بخش 2 → ص 3 · بخش 3 → ص 6 · بخش 4 → ص 7 · بخش 5 → ص 7 · بخش 6 → ص 10 · بخش 7 → ص 12 · بخش 8 → ص 13 · بخش 9 → ص 15 · بخش 10 → ص 19

## چیت‌شیت سریع [CHEAT]

| مفهوم                   | تعریف یک خطی (همین را بنویس)                    |
|-------------------------|-------------------------------------------------|
| Shingle                 | دنباله k حرف/کلمه پشت‌سرهم در سند               |
| Jaccard                 | اندازه اشتراک دو مجموعه ÷ اندازه اجتماع         |
| MinHash                 | احتمال برابر شدن hash دو مجموعه = Jaccard       |
| LSH                     | فقط جفت‌های احتمالاً مشابه را candidate می‌کند  |
| Collaborative Filtering | توصیه بر اساس کاربران یا آیتم‌های شبیه          |
| Closed itemset          | پرتکرار؛ هیچ ابرمجموعه‌ای با همان support ندارد |
| Maximal itemset         | پرتکرار؛ هیچ ابرمجموعه پرتکراری ندارد           |
| MapReduce               | پردازش موازی با Map و Reduce + تحمل خرابی       |

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

که در آن:

$f_{ij}$  = تعداد تکرار کلمه  $i$  در سند  $j$

$\max_k(f_{kj})$  = بیشترین تکرار هر کلمه در همان سند  $j$

$N$  = تعداد کل اسناد

$n_i$  = تعداد اسنادی که کلمه  $i$  در آن‌ها آمده

$$J(A, B) = |A \cap B| / |A \cup B|$$

$$\Pr[h\pi(C_1) = h\pi(C_2)] = J(C_1, C_2)$$

$$P(\text{candidate}) = 1 - (1 - s^r)^b$$

$$L_2 = \sqrt{\sum (x_i - y_i)^2} \quad L_1 = \sum |x_i - y_i|$$

$$L_\infty = \max |x_i - y_i| \quad \text{اسمی: } d = (p - m) / p$$

$$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$$

$$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

$$M \subseteq C \subseteq F$$

$$\text{Map: } (i, m_{ij} \times v_j) \quad \text{Reduce: } u_i = \sum_j (m_{ij} \times v_j)$$

| مقایسه              | نکته طلایی                                      |
|---------------------|-------------------------------------------------|
| RDBMS ↔ Cluster     | عمودی/ACID ↔ افقی/commodity و انتقال کد به داده |
| K-Means ↔ K-Medoids | میانگین (حساس به پرت) ↔ نقطه واقعی داده         |
| Combiner            | فقط اگر Reduce انجمنی و جابجایی‌پذیر باشد       |

## بخش 1 — مقدمه Big Data و TF-IDF [S1]

سه V

• **Volume:** حجم خیلی زیاد

• **Velocity:** تولید با سرعت بالا

• **Variety:** شکل‌های مختلف داده

**داده‌کاوی:** استخراج الگو/دانش مفید از داده.

**دو تکنیک:** خلاصه‌سازی (PageRank، خوشه‌بندی) و استخراج ویژگی (مشابهت، frequent itemsets، توصیه‌گر).

**PageRank:** صفحه مهم است اگر از صفحات مهم لینک بگیرد.

**اصل Bonferroni:** در داده عظیم، رخداد خیلی نادر هم زیاد دیده می‌شود → خطر False Positive.

**Hash:**  $h(x) = x \bmod B$  | **Index:** بازیابی سریع رکورد | **دیسک:** خیلی کندتر از RAM؛ الگوریتم باید I/O کم کند.

کلمه مفید = در تعداد کمی سند، زیاد تکرار شده. کلماتی مثل «و» یا the که همه جا هستند، مفید نیستند (stop words).

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

## مثال 1 – دقیقاً مثل سوال استاد (قدم به قدم)

فرض: تعداد کل اسناد  $N = 1024$

کلمه  $w$  در 128 سند آمده ( $n = 128$ )

در سند  $z$  این کلمه 10 بار آمده

پرتکرارترین کلمه همان سند 100 بار آمده

1 گام  $TF = 10 / 100 = 0.1$

2 گام  $IDF = \log_2(1024 / 128) = \log_2(8) = 3$

چون  $8 = 2^3$

3 گام  $TF-IDF = 0.1 \times 3 = 0.3$

پاسخ نهایی: 0.3

## مثال 2 – برای فهم بهتر

اگر کلمه‌ای در همه اسناد باشد:  $IDF = \log_2(1) = 0$  بی‌اهمیت.

کلمه «هادوپ» فقط در 1 سند از 3 سند آمده، 8 بار؛  $\max$  همان سند = 20:

$TF = 8/20 = 0.4$

$IDF = \log_2(3/1) \approx 1.58$

$TF-IDF \approx 0.4 \times 1.58 = 0.63$

پاسخ نهایی:  $\approx 0.63$

## Collaborative Filtering

- **User-based**: کاربران شبیه تو چه چیزهایی را پسندیده‌اند؟ همان‌ها را به تو پیشنهاد بده.
- **Item-based**: آیتم‌های شبیه آنچه پسندیده‌ای را پیشنهاد بده.

در درس به Recommendation Engine (مثل Netflix و Amazon) و Behavioral Data اشاره شده.

## بخش 2 – Cluster Computing در برابر RDBMS و Hadoop [S2]

**محدودیت‌های RDBMS**: مقیاس‌پذیری گران، سرعت، تحمل خرابی، پردازش پیشرفته روی ترابایت/پتابایت.

**ویژگی‌های Cluster (حفظ کن)**: High Availability, Fault Tolerance, Data Replication, Load Balancing, Automated Failover, Backup & Recovery, Monitoring, Scalability.

| موضوع       | RDBMS سنتی            | Cluster / Hadoop          |
|-------------|-----------------------|---------------------------|
| سخت‌افزار   | سرور گران             | commodity ارزان           |
| مقیاس       | عمودی (تقویت یک سرور) | افقی (افزودن نود)         |
| تحمل خرابی  | غالباً سخت‌افزاری     | نرم‌افزاری با replication |
| ایده پردازش | داده را پیش کد ببر    | کد را پیش داده ببر        |

## نمونه پاسخ آماده برای امتحان

۱) RDBMS برای داده ساخت یافته و تراکنش ACID عالی است، ولی وقتی داده خیلی بزرگ شود هزینه و سرعت مشکل می شود.

۲) Cluster Computing با سرورهای ارزان، کپی داده و failover خودکار، مقیاس افقی می دهد.

۳) جمله طلایی: در Hadoop به جای کشیدن همه داده به یک ماشین، برنامه را به جایی می فرستیم که داده هست.

**Hadoop:** Scalable + Fault Tolerant | HDFS + MapReduce | NameNode / DataNode / JobTracker / TaskTracker | معمولاً هر chunk سه کپی دارد.

**SQL در برابر NoSQL:** SQL اسکیمای ثابت و ACID؛ NoSQL اسکیمای انعطاف پذیر و مقیاس افقی.

## تکمله A – حفظیات داده کاوی، KDD، مخازن داده و Hadoop [D1]

### داده کاوی در برابر KDD

**داده کاوی** استخراج دانش از حجم زیاد داده است؛ نام های نزدیک آن: استخراج دانش، تحلیل داده/الگو، باستان شناسی داده، لایروبی داده و KDD. باین حال در تعریف فرایندی، داده کاوی فقط **یک مرحله** از KDD است.

1. **Data Cleaning:** حذف نویز، تناقض و داده غلط

2. **Data Integration:** ترکیب چند منبع

3. **Data Selection:** انتخاب داده مرتبط با مسئله

4. **Data Transformation:** تبدیل، تجمیع و نرمال سازی

5. **Data Mining:** اجرای روش استخراج الگو

6. **Pattern Evaluation:** تشخیص الگوهای معتبر و جالب

7. **Knowledge Presentation:** نمایش و مصورسازی دانش

### وظایف داده کاوی: حفظیات مهم

| وظیفه              | تعریف                           | مثال                      |
|--------------------|---------------------------------|---------------------------|
| Descriptive        | توصیف ساختار و روابط موجود      | خوشه بندی، قوانین انجمنی  |
| Predictive         | پیش بینی کلاس یا مقدار ناشناخته | طبقه بندی، رگرسیون        |
| Characterization   | خلاصه ویژگی های یک کلاس هدف     | ویژگی مشتریان وفادار      |
| Discrimination     | مقایسه کلاس هدف با کلاس متضاد   | وفادار در برابر ریزشی     |
| Outlier Analysis   | کشف شیء ناسازگار با الگوی عمومی | تقلب کارت اعتباری         |
| Evolution Analysis | مدل کردن روند تغییر در زمان     | سری زمانی و الگوی دوره ای |

**طبقه بندی:** یادگیری با برچسب؛ مدل می تواند IF-THEN، درخت تصمیم، فرمول یا شبکه عصبی باشد. روش های نام برده در لکچر: Naive Bayes، SVM و KNN.

**خوشه بندی:** بدون برچسب؛ گروه های طبیعی را کشف می کند. هر خوشه بعداً می تواند یک کلاس تلقی شود.

**Outlier:** ممکن است نویز باشد، اما گاهی مهم ترین دانش داده است؛ نباید همیشه حذف شود.

### انواع الگوی پرتکرار

- **Frequent Itemset:** اقلامی که مکرراً باهم رخ می دهند؛ مثل شیر و نان
- **Frequent Subsequence:** دنباله عملیاتی که مکرراً با همان ترتیب رخ می دهد
- **Frequent Substructure:** زیرساختار پرتکرار مثل گراف یا درخت

### چه الگویی «جالب» است؟

قابل فهم، معتبر، بالقوه مفید، جدید و نمایش دهنده دانش باشد. معیارهای عینی مانند Support و Confidence آستانه ای دارند که کاربر تعیین می کند.

### معماری سیستم داده کاوی

1. پایگاه/انبار داده/Web یا مخزن دیگر
2. سرور پایگاه یا انبار داده
3. پایگاه دانش (Knowledge Base)

4. موتور داده‌کاوی
5. مازول ارزیابی الگو
6. واسط کاربر

## انواع مخزن داده

| مخزن                   | تعریف و نکته                                                   |
|------------------------|----------------------------------------------------------------|
| Relational DB          | جدول، سطر/تاپل، ستون/صفت و کلید یکتا                           |
| Data Warehouse         | داده یکپارچه چند منبع؛ پاک‌سازی، تبدیل و بارگذاری؛ مدل چندبعدی |
| Data Cube              | نمای چندبعدی؛ نمونه ابعاد: زمان، کالا، مکان                    |
| Transactional DB       | هر رکورد یک TID و فهرست اقلام دارد                             |
| Temporal               | دارای صفت زمانی                                                |
| Sequence               | رویدادهای مرتب، با یا بدون timestamp                           |
| Time-series            | اندازه‌گیری تکراری یک مقدار در زمان                            |
| Spatial                | GIS/CAD/تصویر؛ نمایش Raster یا Vector                          |
| Text / Multimedia      | متن، تصویر، صدا و ویدئو                                        |
| Heterogeneous / Legacy | ترکیب سامانه‌های مستقل، قدیمی و ناهمگون                        |
| Data Stream            | بسیار بزرگ/نامتناهی، پویا، یک یا چند scan، پاسخ سریع و بلادرنگ |

## چالش‌های داده‌کاوی

- استخراج تعاملی در سطوح مختلف انتزاع و استفاده از دانش پس‌زمینه
- زبان پرس‌وجوی داده‌کاوی، نمایش و مصورسازی نتایج
- داده ناقص، نویزی و تشخیص الگوهای باکیفیت
- کارایی و مقیاس‌پذیری؛ الگوریتم موازی، توزیع‌شده و افزایشی
- انواع داده پیچیده و پایگاه‌های ناهمگون

**سطوح اتصال Mining به DB/DW (فهرست حفظی):** No Coupling, Loose Coupling, Semitight Coupling و Tight Coupling.

## Bonferroni – مثال حفظی هتل

$$P(\text{ملاقات یک جفت در یک روز}) = (0.01 \times 0.01) / 10^{-9} = 10^5$$

$$E(\text{دو ملاقات تصادفی}) \approx C(10^9, 2) \times C(1000, 2) \times 10^{-18} \approx 250,000$$

نتیجه: حتی با داده کاملاً تصادفی، حدود ۲۵۰ هزار جفت «مشکوک» پیدا می‌شود؛ پس در داده عظیم، نادر بودن به‌تنهایی دلیل معناداری نیست.

## Hash، Index و دیسک – جزئیات

- تابع هش خوب کلیدها را تقریباً یکنواخت بین B باکت توزیع می‌کند.
- برای  $h(x) = x \bmod B$  معمولاً B اول انتخاب می‌شود تا الگوهای ورودی توزیع را خراب نکنند.
- برخورد هش یعنی چند کلید در یک باکت؛ پس بعد از یافتن باکت باید مقدار واقعی بررسی شود.
- دیسک بلوکی است؛ بلوک کوچک‌ترین واحد انتقال دیسک  $\leftrightarrow$  RAM است.
- خواندن بلوک حدود 10ms و تقریباً  $10^5$  برابر خواندن کلمه از RAM کندتر است.

## Hadoop – جزئیات حفظی

- **فلسفه:** commodity server ارزان + تحمل خرابی نرم‌افزاری + انتقال کد به داده
- **اکوسیستم:** HDFS، MapReduce، HBase، Hive و Pig
- **NameNode:** metadata و محل بلوک‌ها؛ **DataNode:** خود بلوک‌ها
- **JobTracker:** زمان‌بندی Job؛ **TaskTracker:** اجرای Map/Reduce (معماری Hadoop 1)
- **Standalone:** فایل محلی و یک JVM

• **Pseudo-distributed**: یک ماشین ولی HDFS/daemon جدا

• **Fully distributed**: چند نود و replication

• **Cloud**: AWS/S3 و Azure HDInsight/Blob Storage

• **نامناسب**: پایگاه تراکنشی با update بسیار مکرر

**کاربردهای تجاری**: Ad Targeting و Risk Modeling, Customer Churn, Recommendation Engine

**JVM**: Java به bytecode کامپایل می‌شود؛ هر process جاوا JVM و حافظه مجزا دارد. فایل class مستقل از سیستم‌عامل است ولی JVM پلتفرم وابسته است.

### بخش 3 — MapReduce [S3]

تو فقط دو تابع می‌نویسی؛ سیستم بقیه کار را می‌کند.

1. **Map**: هر تکه ورودی را بخوان و جفت‌های (کلید، مقدار) بساز.

2. **Shuffle**: همه مقدارهایی که کلید یکسان دارند را در یک لیست بگذار.

3. **Reduce**: برای هر کلید، لیست مقدارها را ترکیب کن (جمع، شمارش، ...).

**Map: input → (key, value)**

**Shuffle: (k,v1), (k,v2), ... → (k, [v1, v2, ...])**

**Reduce: (k, [v1, v2, ...]) → (k, result)**

#### مثال خیلی ساده — شمارش کلمه

دو جمله داریم: «big data» و «big data systems»

Map 1 جمله: (big,1) (data,1)  
Map 2 جمله: (big,1) (data,1) (systems,1)

بعد از Shuffle:  
big → [1, 1]  
data → [1, 1]  
systems → [1]

Reduce (جمع):  
big = 2  
data = 2  
systems = 1

**پاسخ نهایی: big=2 , data=2 , systems=1**

**Combiner**: اگر عمل Reduce جابجایی‌پذیر و انجمنی باشد (مثل جمع)، می‌توان قبل از شبکه یک Reduce محلی زد تا ترافیک کم شود.

| اگر این خراب شود | چه کار می‌کنند؟                              |
|------------------|----------------------------------------------|
| Master           | معمولاً کل Job از نو شروع می‌شود             |
| Map Worker       | همان Map ها روی نود سالم دوباره اجرا می‌شوند |
| Reduce Worker    | Reduce دوباره اجرا می‌شود                    |

**Workflow Systems**: به جای فقط Map→Reduce، یک گراف DAG از توابع (Clustera, Hyracks).

**Pregel**: سیستم محاسبات گرافی با SuperStep و Checkpoint برای تحمل خرابی در الگوریتم‌های بازگشتی.

**PageRank** تکراری:

تا وقتی تغییر خیلی کم شود  $v^{(t+1)} = M \times v^{(t)}$

#### بخش 4 – جبر رابطه‌ای با MapReduce [S4]

| عملیات              | ایده ساده                                            |
|---------------------|------------------------------------------------------|
| Selection           | فقط تاپل‌هایی که شرط دارند را نگه دار                |
| Projection          | بعضی ستون‌ها را بردار؛ تکراری‌ها را در Reduce حذف کن |
| Union               | همه را بفرست؛ تکراری یکی شود                         |
| Intersection        | فقط چیزی که در هر دو آمده                            |
| Difference R-S      | چیزی که فقط در R است                                 |
| Join                | کلید = ستون مشترک؛ دو طرف را به هم بچسبان            |
| Group + Aggregation | کلید = گروه؛ Reduce = SUM/COUNT/AVG                  |

##### مثال Join برای مبتدی

جدول R: (علی، تهران) و (مینا، اصفهان)  
 جدول S: (تهران، ایران) و (اصفهان، ایران)  
 می‌خواهیم روی «شهر» Join کنیم:

کلید = شهر  
 Map: شهر → از  
 ایران S: علی و از R: تهران → از  
 ایران S: مینا و از R: اصفهان → از  
 تهران: (علی، تهران، ایران) Reduce  
 اصفهان: (مینا، اصفهان، ایران) Reduce

پاسخ نهایی: (علی، تهران، ایران) و (مینا، اصفهان، ایران)

**Multi-way Join:** Join همزمان چند جدول. هزینه انتقال داده مهم است؛ Replication Rate نشان می‌دهد هر ورودی چند بار کپی می‌شود.

#### بخش 5 – ضرب ماتریس با MapReduce [S5]

هدف ماتریس  $\times$  بردار: برای هر سطر ماتریس یک عدد بساز = ضرب داخلی آن سطر در بردار.

$$u_i = m_{i1} \cdot v_1 + m_{i2} \cdot v_2 + \dots + m_{in} \cdot v_n$$

Map:  $(i, m_{ij} \times v_j)$

Reduce: همه مقادیر را جمع کن  $i$  برای هر

## مثال کامل – انگار اولین بار می‌بینی

ماتریس و بردار:

$$M = \begin{matrix} & 1 & 0 & 5 \\ & 7 & 3 & 0 \\ & 1 & 3 & 5 \end{matrix} \quad V = \begin{matrix} 2 \\ 4 \\ 6 \end{matrix}$$

**گام Map:** هر خانه ماتریس را در عنصر متناظر بردار ضرب کن. کلید = شماره سطر.

سطر 1:  $(2=2 \times 1, 1)$  ,  $(0=4 \times 0, 1)$  ,  $(30=6 \times 5, 1)$   
سطر 2:  $(14=2 \times 7, 2)$  ,  $(12=4 \times 3, 2)$  ,  $(0=6 \times 0, 2)$   
سطر 3:  $(2=2 \times 1, 3)$  ,  $(12=4 \times 3, 3)$  ,  $(30=6 \times 5, 3)$

**گام Shuffle:** مقدارهای هر سطر را کنار هم بگذار.

کلید 1  $\rightarrow [30, 0, 2]$   
کلید 2  $\rightarrow [0, 12, 14]$   
کلید 3  $\rightarrow [30, 12, 2]$

**گام Reduce:** جمع کن.

$$\begin{aligned} u_1 &= 2+0+30 = 32 \\ u_2 &= 14+12+0 = 26 \\ u_3 &= 2+12+30 = 44 \end{aligned}$$

پاسخ نهایی:  $U = [32, 26, 44]$

ماتریس  $\times$  ماتریس (دو Job)

$$p_{ik} = \sum_j (m_{ij} \times n_{jk})$$

Job1: روی  $j$  جوین کن و ضرب‌های جزئی بساز. Job2: برای هر  $(i,k)$  جمع بزن.

چک یک خانه

$$\text{از } P: 1 \times 2 + 0 \times 4 + 5 \times 6 = 2+0+30 = 32 \text{ خانه } (1,1)$$

## تکمله B – اجرای داخلی و توسعه‌های MapReduce [D2]

### Grouping و Map Task, Partitioning

- عنصر ورودی (تاپل/سند) نباید بین دو chunk نصف شود.
- Map می‌تواند برای یک ورودی صفر، یک یا چند زوج و حتی چند کلید یکسان تولید کند.
- اگر  $r$  کاهنده باشد،  $h(k)$  یکی از 0 تا  $r-1$  را انتخاب می‌کند.
- هر Map برای هر Reduce یک فایل محلی میانی دارد؛ بنابراین  $r$  فایل می‌سازد.
- شرط اصلی: کلیدهای یکسان حتماً به یک Reduce بروند؛ کلیدهای متفاوت می‌توانند هم‌مقصد شوند.

$$k_1 = k_2 \Rightarrow h(k_1) = h(k_2)$$

$$(k, v_1), (k, v_2), \dots \Rightarrow (k, [v_1, v_2, \dots])$$

**Grouping سیستمی** را با GROUP BY رابطه‌ای اشتباه نکن: Grouping فقط مقادیر هم‌کلید را کنار هم قرار می‌دهد؛ Aggregation عملی است که کاربر در Reduce می‌نویسد.



$$(a \circ b) \circ c = a \circ (b \circ c) \text{ (Associative)}$$

$$a \circ b = b \circ a \text{ (Commutative)}$$

Combiner جای Reduce نهایی را نمی‌گیرد؛ فقط تکراری‌ها/مقادیر همان Map را ترکیب می‌کند. مثلاً در Projection تکراری‌های دو Map مختلف فقط در Reduce نهایی حذف می‌شوند.

### عملیات رابطه‌ای – الگوریتم دقیق

| عملیات             | Map                                        | Reduce                      |
|--------------------|--------------------------------------------|-----------------------------|
| Selection $\sigma$ | اگر $C(t): \text{emit}(t,t)$               | عبور؛ اغلب Map-only         |
| Projection $\pi$   | $\text{emit}(t',t')$                       | یک $t'$ برای حذف تکرار      |
| Union              | از $R$ و $S: \text{emit}(t,t)$             | یک بار $t$                  |
| Intersection       | $R: \text{emit}(t,R), S: \text{emit}(t,S)$ | اگر هر دو برچسب بودند       |
| Difference $R-S$   | همراه برچسب منبع                           | فقط $\text{labels}=\{R\}$   |
| Join               | کلید=همه صفات مشترک                        | حاصل ضرب دکارتی محلی دو طرف |
| Grouping/Agg       | کلید=صفات گروه                             | SUM/COUNT/AVG/MIN/MAX       |

**تفاضل جابجایی‌پذیر نیست:**  $R-S$  با  $S-R$  فرق دارد. در Intersection حتماً نام رابطه را بفرست؛ دو تکرار از یک رابطه به معنی حضور در هر دو رابطه نیست.

**Join:** اگر برای یک کلید، ۲ تاپل  $R$  و ۳ تاپل  $S$  باشد، Reduce باید  $6=3 \times 2$  خروجی بسازد.

### Matrix×Vector وقتی بردار در RAM جا نمی‌شود

ماتریس را به نوارهای عمودی و بردار را به قطعات افقی متناظر تقسیم کن؛ تعداد نوارها طوری باشد که قطعه بردار و chunk ماتریس در حافظه یک نود جا شود. هر Map سهم جزئی  $u_i$  را می‌سازد و Reduce سهم نوارهای مختلف را روی  $i$  جمع می‌کند.

### Matrix×Matrix: دو Job در برابر یک Job

**دو Job:** روی  $J$  جوی  $n$  و  $(m_{ij}n_{jk}, (i,k))$  تولید می‌کند؛ Job2 روی  $(i,k)$  جمع می‌زند.

**یک Job:** هر  $m_{ij}$  برای تمام  $k$ ها و هر  $n_{jk}$  برای تمام  $i$ ها تکثیر می‌شود. Job کمتر، ولی Replication و ترافیک بیشتر.

### بستار تعدی (Transitive Closure)

$$P = E \cup \{ (x,y) \mid \exists z : P(x,z) \wedge P(z,y) \}$$

اگر مسیر  $(1,2)$  و  $(2,3)$  داشته باشیم، Join روی گره میانی 2 مسیر  $(1,3)$  را می‌سازد. دو نوع Task: Join Task و Duplicate-Elimination Task. مسیر جدید پس از حذف تکراری به Join Taskهای مربوط به دو سر مسیر فرستاده می‌شود؛ توقف وقتی مسیر جدیدی ساخته نشود.

### Pregel

- اجرای همگام در مرحله‌های SuperStep
- Checkpoint = کپی وضعیت همه Taskها
- در خرابی، بازگشت به آخرین Checkpoint
- زمان ترمیم باید خیلی کمتر از میانگین زمان بین دو خرابی باشد

### Replication Rate و Communication Cost، Reducer Size

$$\text{Communication Cost} = \sum |\text{Input هر Task}|$$

$$\rho = (\text{تعداد عناصر ورودی}) / (\text{Map تعداد کل خروجی‌های})$$

**Reducer Size (q):** بیشترین تعداد مقدار مجاز برای یک q. Reduce کوچکتر → Reduce های بیشتر و موازی سازی بیشتر؛ اگر ورودی Reduce در RAM جا شود I/O کمتر می شود.

**Trade-off:** یک Reduce هزینه انتقال حداقل ولی زمان اجرا زیاد؛ Reduce های بیشتر زمان کمتر ولی احتمال تکثیر/هزینه بیشتر.

### Multi-way Join یک مرحله ای

برای  $R(A,B) \bowtie S(B,C) \bowtie T(C,D)$ ، تعداد باکت B برابر b و C برابر c است؛ پس  $k=bc$  کاهنده.

- $S(v,w)$  فقط به Reduce با آدرس  $(h(v),g(w))$  می رود.
- $R(u,v)$  چون C ندارد به تمام C ستون تکثیر می شود.
- $T(w,x)$  چون B ندارد به تمام b سطر تکثیر می شود.

$$CC_{reduce} = s + c \cdot r + b \cdot t \quad \text{با شرط } b \cdot c = k$$

$$b^* = \sqrt{(k \cdot r / t)} \quad c^* = \sqrt{(k \cdot t / r)}$$

$$CC_{reduce}(\min) = s + 2\sqrt{(krt)}$$

$$CC_{total} = r + 2s + t + 2\sqrt{(krt)}$$

### مثال Multi-way Join

اگر  $|R|=|T|=100$  و  $k=4$ ، انتخاب بهینه  $b=c=2$  است.

$$CC_{reduce} = s + 2 \times 100 + 2 \times 100 = s + 400$$

\n یک بار S دو بار T دو بار R هر

**روش دو مرحله ای:** اگر احتمال تطابق R و S برابر p باشد، اندازه داده میانی تقریباً prs و هزینه  $O(r+s+t+prs)$  است. انتخاب روش به اندازه روابط و خروجی میانی بستگی دارد.

### بخش 6 — Shingle ، MinHash ، LSH [S6]

**Document → Shingles → MinHash Signature → LSH → Candidates → Exact check**

### Shingle چیست؟

یک پنجره به طول k روی متن می گذاری و هر بار یک تکه برمی داری.

### مثال

متن: abcab و  $k = 2$

تکه ها: ab , bc , ca , ab  
مجموعه بدون تکرار: {ab, bc, ca}

### Jaccard

$$J(A,B) = |A \cap B| / |A \cup B|$$

### مثال با میوه

$A = \{\text{سیب، پرتقال، موز}\}$  و  $B = \{\text{موز، انگور}\}$

$1 \rightarrow$  اشتراک  $\{\text{موز}\} =$   
 $4 \rightarrow$  اجتماع  $= \{\text{سیب، پرتقال، موز، انگور}\}$   
 $J = 1/4 = 0.25$   
فاصله  $= 1 - 0.25 = 0.75$

پاسخ نهایی:  $J = 0.25$

### MinHash

عناصر را تصادفی مرتب کن. برای هر مجموعه، اولین عنصری که در آن ترتیب می‌بینی را به عنوان hash بردار.

همان دو مجموعه Jaccard = [ هاش دو مجموعه برابر شود ] Pr

### مثال

$A = \{1,3,4\}$  و  $B = \{2,3,4,5\}$

واقعی  $Jaccard = 2/5 = 0.4$

یک ترتیب تصادفی فرضی: 3, 1, 5, 2, 4

در این ترتیب  $A$  3 اولین عنصر  
در این ترتیب  $B$  3 اولین عنصر  
ها برابر شدند hash پس

اگر خیلی ترتیب تصادفی بگیری، نسبت دفعاتی که برابر می‌شوند حدود 0.4 می‌شود.

### LSH

امضا را به چند نوار (band) تقسیم کن. اگر دو سند حتی در یک نوار کاملاً یکسان بودند، آن‌ها را candidate حساب کن و بعد دقیق چک کن.

$$P = 1 - (1 - s^r)^b$$

که در آن:

$s$  = شباهت واقعی

$r$  = تعداد سطر در هر band

$b$  = تعداد bandها

### مثال عددی

$b = 20, r = 5, s = 0.8$

$s^r = 0.8^5 \approx 0.328$   
 $(1 - 0.328)^{20} = 0.672^{20} \approx 0.00035$   
 $P = 1 - 0.00035 \approx 0.99965$

یعنی حدود 99.97٪ احتمال دارد جفت واقعاً مشابه را پیدا کنیم.

پاسخ نهایی:  $P \approx 0.99965$

**PCY:** در گذر اول علاوه بر شمارش ارقام، جفت‌ها را به bucket هاش کن. در گذر دوم فقط جفتی را بشمار که هر دو قلم frequent باشند و bucketشان هم frequent باشد.

چهار شرط Metric: نامنفی، صفر فقط وقتی دو شیء یکی باشند، تقارن، نامساوی مثلثی.

$$L_2 \text{ (اقلیدسی)} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots}$$

$$L_1 \text{ (منهتن)} = |x_1 - y_1| + |x_2 - y_2| + \dots$$

$$L_\infty = \text{بزرگ‌ترین } |x_i - y_i|$$

مثال روی یک جفت نقطه

$Y = (4, 6)$  و  $X = (1, 2)$

$$L_2 = \sqrt{(1-4)^2 + (2-6)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

$$L_1 = |1-4| + |2-6| = 3 + 4 = 7$$

$$L_\infty = \max(3, 4) = 4$$

$$d = (p - m) / p \text{ : برای صفات اسمی}$$

که در آن:

$p$  = تعداد کل صفات

$m$  = تعداد صفاتی که مقدارشان یکی است

مثال اسمی

شخص 1: (مرد، تهران، لیسانس)

شخص 2: (زن، تهران، لیسانس)

$$p = 3$$

$$m = 2 \text{ (شهر و مدرک یکسان اند)}$$

$$d = (3-2)/3 = 1/3 \approx 0.33$$

پاسخ نهایی: 1/3

دودویی:  $q = \text{هر دو } 1 \mid t \mid s = (1,0) \mid r = (0,1) = \text{هر دو } 0$

$$d = (r + s) / (q + r + s + t) \text{ : متقارن}$$

$$d = (r + s) / (q + r + s) \text{ : نامتقارن}$$

$$\text{Jaccard: } J = q / (q + r + s)$$

**Cosine** برای متن خوب است. **Hamming** = تعداد خانه‌های متفاوت. **Edit distance** =  $|X| + |Y| - 2 \times |LCS|$ .

**Data Matrix**: سطر = تاپل، ستون = صفت. **Dissimilarity Matrix**: فاصله بین تاپل‌ها؛ قطر اصلی صفر.

$\text{support}(X) = (\text{تعداد تراکنش شامل } X) / (\text{کل تراکنش‌ها})$

$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$

$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$

$M \subseteq C \subseteq F$

#### مثال قانون انجمنی

از 100 سبد خرید: نان در 40 سبد، نان+شیر در 30 سبد.

قانون: نان  $\Rightarrow$  شیر

$\text{support} = 30/100 = 0.3$

$\text{confidence} = 30/40 = 0.75 = 75\%$

یعنی 75٪ کسانی که نان گرفته‌اند شیر هم گرفته‌اند.

پاسخ نهایی:  $\text{conf} = 75\%$

| نوع     | معنی ساده                                               |
|---------|---------------------------------------------------------|
| Closed  | اگر قلمی اضافه کنی، support دیگر همان عدد قبلی نمی‌ماند |
| Maximal | اگر قلمی اضافه کنی، دیگر frequent نیست                  |

#### مثال Closed و Maximal

تراکنش‌ها:  $\{A,B\}$  ،  $\{A,B,C\}$  ،  $\{A,C\}$  و حداقل شمارش  $= 2$

$A=4$  ،  $B=3$  ،  $C=3$

$AB=3$  ،  $AC=3$  ،  $BC=2$

$ABC=2$

Closed ها:  $A$  ،  $AB$  ،  $AC$  ،  $ABC$

(را دارد  $\text{support}=3$  همان  $AB$  نیست چون  $B$  closed)

Maximal فقط:  $ABC$

**اصل Apriori:** اگر مجموعه‌ای frequent باشد، همه زیرمجموعه‌هایش هم frequent اند. پس کاندید بزرگ را فقط از frequent های کوچک می‌سازیم.

**CHARM:** الگوریتم استخراج Closed با اشتراک tidset ها.

### تکمله C — جزئیات CHARM و Similarity, MinHash, LSH, PCY [D3]

#### Characteristic Matrix

- هر سطر = یک shingle یکتا؛ هر ستون = یک سند
- خانه 1 یعنی shingle در سند وجود دارد؛ ماتریس معمولاً بسیار sparse است.
- AND دو ستون = اشتراک؛ OR = اجتماع.

**Set در برابر Bag:** Set تکرار shingle را حذف می‌کند؛ Bag فراوانی را نگه می‌دارد. MinHash کلاسیک روی Set است.

#### انتخاب k و Hash کردن Shingle

- $k$  خیلی کوچک  $\rightarrow$  اسناد نامرتبط هم اشتراک زیاد دارند.
- در لکچر:  $k \approx 5$  برای سند کوتاه و  $k \approx 10$  برای سند بلند.
- می‌توان shingle را به شناسه 4 بایتی hash کرد تا فضا کم شود.

• Collision می‌تواند شباهت را کمی بیش‌برآورد کند؛ فضای hash باید بزرگ باشد.

### اثبات MinHash – جمله‌ای که حفظ می‌کنی

در اجتماع  $C_1 \cup C_2$ ، نخستین عضو permutation با احتمال مساوی هر عضو است. دو MinHash فقط وقتی برابرند که نخستین عضو از اشتراک آمده باشد؛ بنابراین احتمال  $| اشتراک | / | اجتماع |$ .

$$\hat{J}(C_1, C_2) = (\text{Signature تعداد سطرهای برابر دو}) / K$$

$$E[\hat{J}] = J \text{ واقعی}$$

هرچه تعداد توابع K بیشتر باشد، تخمین پایدارتر ولی حافظه/زمان بیشتر می‌شود.

### الگوریتم یک‌گذری MinHash بدون permutation

1. همه خانه‌های Signature را  $\infty$  بگذار.
2. سطرهای ماتریس مشخصه را یک‌بار scan کن.
3. اگر سند C در سطر j مقدار 1 داشت، برای هر تابع i بنویس:  $\text{sig}(C)[i] = \min(\text{sig}(C)[i], h_i(j))$ .

$$h_{a,b}(x) = ((a \cdot x + b) \bmod p) \bmod N \quad (p > N \text{ اول})$$

### LSH: منطق AND/OR و S-Curve

- داخل هر band: AND روی r سطر؛ همه باید برابر باشند.
- بین bandها: OR؛ برابری فقط یک band کافی است.
- منحنی احتمال نسبت به S شکل S دارد؛ آستانه تقریبی  $(1/r)^{(b/1)}$ .
- r بیشتر: FP کمتر، FN بیشتر. b بیشتر: FN کمتر، FP بیشتر.
- Candidate بودن یعنی «ارزش بررسی دقیق دارد»، نه اینکه قطعاً مشابه است.
- Exact Check، FP را حذف می‌کند؛ FN از دست‌رفته را نمی‌تواند برگرداند.

نشده candidate جفت واقعاً مشابه که False Negative =

شده و بعداً رد می‌شود candidate جفت نامشابه که False Positive =

### Mixed Attributes و Normalizing

$$\text{Min-Max: } z_{if} = (x_{if} - \min(xf)) / (\max(xf) - \min(xf))$$

$$\text{Z-score: } z_{if} = (x_{if} - \mu_f) / \sigma_f$$

$$\text{Mixed: } d(i, j) = \sum \delta_{ij}^f d_{ij}^f / \sum \delta_{ij}^f$$

$\delta=0$  اگر مقدار missing باشد یا صفت دودویی نامتقارن و هر دو صفر باشند. صفت ترتیبی با M مرتبه ابتدا به  $(rank-1)/(M-1)$  تبدیل و سپس مثل عددی محاسبه می‌شود.

**هشدار:**  $\cosine-1$  یک dissimilarity رایج است، ولی لزوماً Metric نیست. فاصله زاویه‌ای Metric  $\arccos(\cosine)$  مناسب‌تری است. فرمول LCS فقط برای درج/حذف است، نه Levenshtein عمومی که جایگزینی هم دارد.

### PCY – جزئیات Bitmap

1. Pass1: تک‌اقلام را دقیق بشمار و همه جفت‌های هر basket را به B bucket hash کن.
  2. بین دو bucket: passهای با  $\text{count} \geq \text{minsup}$  را در bitmap علامت بزن.
  3. Pass2: جفت فقط اگر هر دو item frequent و bit باکت =1 باشد شمارش شود.
- باکت frequent تضمین نمی‌کند خود جفت frequent باشد (collision)، اما باکت infrequent قطعاً نمی‌تواند جفت frequent داشته باشد.

$$t(X) = \{ \text{TID های } X \text{ هست که TID} \}$$

$$i(Y) = \{ \text{TID های } Y \text{ اقلام مشترک در همه} \}$$

$$\text{closure: } c(X) = i(t(X))$$

$$X \text{ بسته است} \Leftrightarrow c(X) = X$$

در کلاس itemset های با tidset یکسان، بزرگترین itemset همان Closed نماینده است. Closed یک نمایش lossless برای support است؛ Maximal فشرده تر است ولی support زیرمجموعه ها را از دست می دهد.

### CHARM

- نمایش عمودی itemset × tidset و prefix-equivalence class
- $t(X \cup Y) = t(X) \cup t(Y)$  = support این اشتراک
- تولید کاندید و شمارش support هم زمان است.
- tidset برابر  $\rightarrow$  اقلام همیشه باهم اند؛ در closure ادغام شوند.
- رابطه زیرمجموعه tidset ها برای هرس شاخه ها استفاده می شود.

### قواعد Non-redundant

از Closed ها یک basis کوچک از قواعد می سازیم که سایر قواعد قابل استنتاج باشند. قواعد  $\text{confidence} = 100\%$  از closed بزرگ تر به کوچک ترند؛ قواعد کمتر از  $100\%$  به closed superset حرکت می کنند.

#### مثال های منبع

$CW \Rightarrow C$  :  $\text{support} = 5/6$  ,  $\text{confidence} = 5/5 = 1$  \n  $CW \Rightarrow ACW$  :  $\text{support} = 3/6 = 0.5$  ,  $\text{confidence} = 3/5 = 0.6$

## بخش 9 — خوشه بندی [S9]

**هدف:** اعضای یک خوشه به هم نزدیک؛ خوشه های مختلف از هم دور. یادگیری بدون سرپرست (برخلاف طبقه بندی).  
انواع: Partitioning (K-Means, K-Medoids, PAM, CLARA, CLARANS) · Hierarchical (AGNES, DIANA) · Density Grid (STING, CLIQUE) · (DBSCAN, OPTICS, DENCLUE)

### مثال K-Means — فقط یک تکرار

نقاط:  $A(1,1)$  ,  $B(1,2)$  ,  $C(4,4)$  ,  $D(5,4)$

مراکز اولیه:  $\mu_1 = A(1,1)$  و  $\mu_2 = C(4,4)$

**تخصیص:** هر نقطه به نزدیک ترین مرکز می رود.

$C_1 = \{A, B\}$   $\rightarrow$  هستند  $\mu_1$  نزدیک B و A

$C_2 = \{C, D\}$   $\rightarrow$  هستند  $\mu_2$  نزدیک D و C

**به روز رسانی مرکز:** میانگین بگیر.

$\mu_1 = ((1+1)/2, (1+2)/2) = (1, 1.5)$

$\mu_2 = ((4+5)/2, (4+4)/2) = (4.5, 4)$

**پاسخ نهایی:** مراکز جدید:  $(1.5, 1)$  و  $(4, 4.5)$

**Linkage:** Single = کمینه فاصله بین دو خوشه · Complete = بیشینه · Average = میانگین · Centroid = فاصله مراکز.  
**K-Medoids / PAM:** نماینده خوشه یک نقطه واقعی است (مقاوم تر به پرت). CLARA روی نمونه کار می کند؛ CLARANS جستجوی تصادفی می کند.

**DBSCAN: Core Point** → نقطه باشد MinPts حداقل Eps اگر در شعاع  
**Border** = Core است ولی خودش کنار  
**Noise** = هیچکدام

**BFR (داده بزرگ اقلیدسی):**  
 $\text{centroid}_i = \text{SUM}_i / N$   
 $\text{variance}_i = \text{SUMSQ}_i / N - (\text{SUM}_i / N)^2$

**CURE:** چند نقطه نماینده برای خوشه‌های غیرکروی. **OPTICS:** وقتی چگالی خوشه‌ها فرق می‌کند، بهتر از DBSCAN است.

## تکمله D – جزئیات کامل الگوریتم‌های خوشه‌بندی [D4]

### کاربردها و نیازها

کاربرد: بخش‌بندی مشتری، بازیابی اطلاعات، زیست‌شناسی، داده فضایی، امنیت، شناسایی outlier، پیش‌پردازش، جست‌وجوی وب و خلاصه‌سازی.  
 نیازها: مقیاس‌پذیری، انواع صفت، شکل دلخواه، دانش قبلی کم، تحمل نویز، افزایشی و مستقل از ترتیب، ابعاد بالا، محدودیت کاربر و تفسیرپذیری.

### خوشه‌بندی سلسله‌مراتبی

| روش   | حرکت       | شرح                                  |
|-------|------------|--------------------------------------|
| AGNES | پایین→بالا | هر نقطه یک خوشه؛ نزدیک‌ترین‌ها ادغام |
| DIANA | بالا→پایین | همه یک خوشه؛ مرتب تقسیم              |

خروجی معمولاً **Dendrogram** است. سه تصمیم: نمایش خوشه، فاصله دو خوشه، و شرط توقف.

### Centroid در برابر Clustroid

$\text{Centroid } \mu_C = (1/|C|) \sum_{x \in C} x$   
**Clustroid** = یک نقطه واقعی که مجموع/بیشینه فاصله تا بقیه را کمینه  
 کند

Centroid مناسب فضای اقلیدسی و ممکن است عضو واقعی داده نباشد. در اسناد/رشته‌ها «میانگین» معنی ندارد، پس clustroid یا medoid می‌گیریم.



$$\text{Single}(C_1, C_2) = \min d(x, y)$$

$$\text{Complete}(C_1, C_2) = \max d(x, y)$$

$$\text{Average}(C_1, C_2) = [\sum d(x, y)] / (|C_1||C_2|)$$

$$\text{radius}(C) = \max d(x, \mu_C)$$

$$\text{diameter}(C) = \max d(x, y)$$

- Single: مناسب شکل زنجیره‌ای، ولی اثر chaining و پل نویزی دارد.
  - Complete: خوشه فشرده، ولی حساس به outlier.
  - Average: تعادل بین دو روش.
- توقف: رسیدن به  $k$ ، عبور radius/diameter از آستانه، افت چگالی، جهش کیفیت یا یک خوشه.

### K-Means – نکات امتحانی

$$\text{SSE} = \sum_j \sum_{x \in C_j} ||x - \mu_j||^2$$

- هر مرحله SSE را زیاد نمی‌کند، پس همگرا می‌شود؛ اما به بهینه محلی.
- مقداردهی بهتر: مراکز دور از هم، نمونه کوچک، یا چند اجرا و انتخاب کمترین SSE.
- انتخاب Elbow  $k$ : جایی که افزایش  $k$  دیگر بهبود زیاد نمی‌دهد.
- ضعف: نیاز به  $k$ ، حساس به init/outlier، نامناسب شکل غیرکروی و صفت اسمی.

### چرا Outlier بد است؟

با اضافه شدن 100: میا نگین = 26.5 \n میا نگین نقاط {1, 2, 3} = 2

### K-Medoids, PAM, CLARA, CLARANS

- **K-Medoids**: نماینده نقطه واقعی؛ فاصله دلخواه؛ مقاوم‌تر ولی گران‌تر.
- **PAM**:  $k$  medoid → تخصیص → امتحان تعویض medoid/non-medoid → قبول اگر هزینه کم شد.
- **CLARA**: چند نمونه تصادفی، PAM روی هر نمونه، ارزیابی روی کل داده؛ سریع ولی وابسته به نمونه.
- **CLARANS**: جواب‌های medoid را گراف می‌بیند و همسایه تصادفی را می‌گردد؛ کیفیت بهتر، هنوز پرهزینه.

### BFR – فرض‌ها و سه مجموعه

فضای اقلیدسی پُریرد؛ خوشه تقریباً نرمال؛ ابعاد مستقل؛ بیضی‌ها محورهم‌راستا. هر خوشه با  $2d+1$  مقدار  $N$ ,  $SUM$ ,  $SUMSQ$  خلاصه می‌شود.

| معنی                                                            | مجموعه               |
|-----------------------------------------------------------------|----------------------|
| با اطمینان عضو خوشه اصلی؛ خود نقاط دور ریخته و خلاصه حفظ می‌شود | DS (Discard Set)     |
| گروه فشرده، ولی هنوز نزدیک هیچ DS نیست                          | CS (Compression Set) |
| نقاط منفرد/مشکوک که صریح نگه داشته می‌شوند                      | RS (Retained Set)    |

$$\text{Mahalanobis}(x, C) = \sqrt{\sum_i [(x_i - \mu_i) / \sigma_i]^2}$$

مراحل: نمونه اولیه و RS → DS → خوشه‌کردن RS و ساخت CS → خواندن chunk → نزدیک‌ها به DS، سپس CS، بقیه RS → ادغام CS‌ها → در chunk آخر ادغام مناسب با DS.

### CURE

1. نمونه تصادفی در RAM

2. خوشه‌بندی سلسله‌مراتبی نمونه
3. انتخاب c نماینده که از هم دور باشند
4. حرکت هر نماینده کسری  $\alpha$  به سمت centroid
5. فاصله خوشه‌ها = کمترین فاصله نماینده‌ها
6. تخصیص کل داده به نزدیک‌ترین نماینده

$$r' = r + \alpha(\mu - r)$$

نماینده‌های متعدد شکل خوشه را می‌گیرند؛ shrink به مرکز اثر outlier مرزی را کم می‌کند.

## DBSCAN – روابط دقیق

$$N_{\text{Eps}}(q) = \{ p \mid \text{dist}(p, q) \leq \text{Eps} \}$$

$$\text{Core} \Leftrightarrow |N_{\text{Eps}}(q)| \geq \text{MinPts}$$

- **Directly density-reachable**: p در همسایگی q و q یک Core باشد؛ لزوماً متقارن نیست.
- **Density-reachable**: زنجیره‌ای از دسترسی‌های مستقیم.
- **Density-connected**: نقطه 0 وجود دارد که p و q هر دو از آن قابل دسترسی‌اند؛ متقارن است.
- با index فضایی تقریباً  $O(n \log n)$ ، بدون آن  $O(n^2)$ .
- Eps کوچک → noise زیاد؛ Eps بزرگ → اتصال خوشه‌های جدا.
- MinPts کم → نویز خوشه؛ MinPts زیاد → خوشه کوچک حذف.

## OPTICS

$$\begin{aligned} \text{coreDist}(p) &= \text{آمین همسایه-MinPts تا } p \text{ فاصله} \\ \text{reachDist}(p, q) &= \max(\text{coreDist}(q), \text{dist}(p, q)) \end{aligned}$$

به جای یک افراز، ترتیب نقاط و Reachability Plot می‌دهد. دره‌ها خوشه‌اند؛ دره عمیق‌تر یعنی چگالی بیشتر. مناسب‌تر از DBSCAN برای چگالی‌های متفاوت.

## CLIQUE و STING

| روش    | ایده                                                        | مزیت                 | ضعف                             |
|--------|-------------------------------------------------------------|----------------------|---------------------------------|
| STING  | شبکه سلسله‌مراتبی؛ در هر سلول count/mean/min/max            | سریع، موازی، افزایشی | مرز پله‌ای/محوره‌م‌راستا        |
| CLIQUE | سلول‌های متراکم در زیرفضای پُر بعد؛ تولید زیرفضا با Apriori | کشف خودکار زیرفضا    | حساس به اندازه grid و threshold |

## راهنمای انتخاب الگوریتم

| شرایط                                | انتخاب    |
|--------------------------------------|-----------|
| عددی، کروی، سریع                     | K-Means   |
| پرت زیاد یا فاصله غیر اقلیدسی        | K-Medoids |
| داده عظیم و خوشه نرمال/محوره‌م‌راستا | BFR       |
| شکل غیر کروی در فضای اقلیدسی         | CURE      |
| شکل دلخواه + نویز                    | DBSCAN    |
| چگالی‌های متفاوت                     | OPTICS    |
| ابعاد بالا و خوشه در بعضی ابعاد      | CLIQUE    |

## سوال 1 – تعاریف

**الف (Min-hashing):** از مجموعه بزرگ امضای کوتاه می‌سازیم طوری که احتمال برابر شدن hash دو مجموعه برابر Jaccard باشد.

$$\Pr[ h(C_1) = h(C_2) ] = J(C_1, C_2)$$

**ب Collaborative Filtering:** توصیه‌گر مبتنی بر شباهت کاربران یا آیتم‌ها (مثل Netflix/Amazon).  
**ج Jaccard:**

$$J(A,B) = |A \cap B| / |A \cup B|$$

**د Shingle:** دنباله k توکن متوالی در سند.

## سوال 2 – Cluster در برابر RDBMS

RDBMS: مقیاس غالباً عمودی، ACID، داده ساخت‌یافته، در حجم خیلی بالا گران/کند.  
Cluster: مقیاس افقی، replication، commodity servers، failover، انتقال کد به سمت داده.

## سوال 3 – MapReduce با Matrix×Vector

$$\text{Map: } (i, m_{ij} \times v_j)$$

$$\text{Reduce: } u_i = \text{مجموع همه مقادیرهای کلید } i$$

مثال عددی کامل در بخش 5.

## سوال 4 – TF-IDF

## حل سوال استاد

$$\begin{aligned} TF &= 10/100 = 0.1 \\ IDF &= \log_2(1024/128) = \log_2(8) = 3 \\ TF-IDF &= 0.1 \times 3 = 0.3 \end{aligned}$$

پاسخ نهایی: 0.3

## سوال 5 – MST = 0.3 با Apriori

| TID | افلام   | TID | افلام         |
|-----|---------|-----|---------------|
| 1   | a, b    | 5   | a, b, c       |
| 2   | b, c    | 6   | d, f          |
| 3   | a, b, c | 7   | c, d, e, f    |
| 4   | d, e, f | 8   | a, b, c, d, e |

### حل قدم به قدم

تعداد تراکنش = 8 و  $\text{minsup} = 0.3 \rightarrow$  حداقل شمارش باید  $\leq 3$  باشد (چون  $2/8 = 0.25$  از 0.3 کمتر است).

#### Pass 1

$a=4, b=5, c=5, d=4, e=3, f=3$   
 $3 \leq \text{همه} \rightarrow L1 = \{a, b, c, d, e, f\}$

#### Pass 2

$ab=4 \checkmark \quad ac=3 \checkmark \quad bc=4 \checkmark$   
 $de=3 \checkmark \quad df=3 \checkmark$   
 $cd=2 \times \quad ce=2 \times \quad ef=2 \times$   
 $L2 = \{ab, ac, bc, de, df\}$

**Pass 3:** فقط  $abc$  قابل ساخت است ( $ab$  و  $ac$  و  $bc$  همه frequent اند).  $def$  نه، چون  $ef$  frequent نیست.

$\text{count}=3 \checkmark \rightarrow$  در تراکنشهای 3 و 5 و 8  
 $L3 = \{abc\}$

Pass 4 کاندیدی ندارد  $\rightarrow$  توقف.

**پاسخ نهایی:**  $L2 = \{ab, ac, bc, de, df\}$  و  $L3 = \{abc\}$

### چک لیست ۳۰ ثانیه‌ای: تعریف یک خطی؟ فرمول؟ مثال عددی؟ تفاوت با مفهوم نزدیک؟

موارد اضافه شده پس از مرور لکچرها: Hash/Index/دیسک، Data Streams، Bonferroni، Pregel، Multi-way Join، PCY، OPTICS/CURE/BFR. Online Advertising فقط در فهرست کلی overview بودند و اسلاید کامل در فایل‌های ارسالی نبود؛ تمرکز نمونه سوالات استاد روی Similar و Apriori و Items، MapReduce، TF-IDF، Cluster/RDBMS است.